

Esame Programmazione di Sistema / API Programming

Data: 2025-09-15

QUESTIONS

Concurrent Cache Implementation

The goal of this exam is to implement a thread-safe cache with automatic expiration of entries.

This trait defines the interface for a key-value store where entries automatically expire after a specified duration. Implementations should maintain thread safety through synchronization primitives and run a background thread to periodically clean up expired entries.

Features:

- **Thread-safe access to cached values:** Concurrent reads and writes must be safe.
- **Automatic expiration of entries:** Entries expire after a specified duration.
- **Background cleanup:** A dedicated thread cleans up expired entries periodically.
- **Proper resource management:** Resources must be properly cleaned up when the cache is dropped.

Thread Safety:

Implementations should be designed to be safely shared across multiple threads. They should use synchronization primitives to ensure thread-safe access to the cached data. Multiple threads should be able to concurrently call `get` and `set` methods without causing data races or other concurrency issues.

Memory Management:

Implementations should automatically manage memory by:

1. Running a background thread that periodically removes expired entries.
2. Properly cleaning up resources when the cache is dropped.

This ensures that memory usage doesn't grow unbounded even with frequent cache operations and that no resources are leaked when the cache is no longer needed.

```

1 mod concurrent_cache {
2     use std::collections::HashMap;
3     use std::sync::{Arc, Condvar, Mutex};
4     use std::thread;
5     use std::thread::{JoinHandle};
6     use std::time::{Duration, Instant};
7
8     pub trait ConcurrentCache {
9         /// Creates a new cache with the specified expiration duration.
10        fn new(d: Duration) -> Self where Self: Sized;
11
12        /// Retrieves a value from the cache if it exists and hasn't
13        expired.
14        fn get(&self, key: &str) -> Option<Arc<String>>;
15
16        /// Stores a value in the cache with an expiration time.
17        fn set(&self, key: &str, value: &str);
18    }
19
20    pub struct ConcurrentCacheImpl {
21        // Add your fields here
22    }

```

SOLUTIONS

Solution Document Content

Overview

This document serves as a placeholder for the step-by-step solution to the exam dated 2025-09-15.

Step 1: Problem Analysis

We need to design a thread-safe key-value store in Rust. The key operations (**get** and **set**) must be synchronized, typically using a **Mutex**. Furthermore, the entries must automatically expire after a specified duration **ttl**. A background thread should periodically scan the map and remove expired elements. To avoid the thread running forever, a **Condvar** should signal the thread to shut down when the cache is dropped.

Step 2: Architecture Design

- **Shared State:** A **Mutex** protecting a **HashMap<String, (Arc<String>, Instant)>** and a **stop** flag.
- **Synchronization:** A **Condvar** is paired with the **Mutex** to allow the background thread to wait efficiently for the next expiration time or for a shutdown signal.
- **Background Thread:** Spawned in **new()**, loops until **stop** is true, waiting on the **Condvar** with a timeout equal to the time until the next entry expires.
- **Graceful Shutdown:** Implemented in the **Drop** trait by setting **stop = true**, notifying the **Condvar**, and joining the thread.

Step 3: Implementation

```

1 use std::collections::HashMap;
2 use std::sync::{Arc, Condvar, Mutex};
3 use std::thread;
4 use std::thread::JoinHandle;
5 use std::time::{Duration, Instant};
6
7 pub trait ConcurrentCache {
8     fn new(d: Duration) -> Self where Self: Sized;
9     fn get(&self, key: &str) -> Option<Arc<String>>;
10    fn set(&self, key: &str, value: &str);
11 }
12
13 struct CacheState {
14     map: HashMap<String, (Arc<String>, Instant)>,
15     stop: bool,
16 }
17
18 pub struct ConcurrentCacheImpl {
19     state: Arc<(Mutex<CacheState>, Condvar)>,
20     ttl: Duration,
21     cleaner_thread: Option<JoinHandle<()>>,
22 }
23
24 impl ConcurrentCache for ConcurrentCacheImpl {
25     fn new(d: Duration) -> Self {
26         let state = Arc::new((
27             Mutex::new(CacheState {
28                 map: HashMap::new(),
29                 stop: false,
30             }),
31             Condvar::new(),
32         ));
33
34         let state_clone = Arc::clone(&state);
35         let cleaner_thread = thread::spawn(move || {
36             let (lock, cvar) = &*state_clone;
37             let mut state = lock.lock().unwrap();
38
39             while !state.stop {
40                 let now = Instant::now();
41                 let mut next_wakeup = now + d;
42
43                 state.map.retain(|_, (_, expiry)| {
44                     if *expiry > now {
45                         if *expiry < next_wakeup {
46                             next_wakeup = *expiry;
47                         }
48                         true
49                     } else {
50                         false
51                     }
52                 });
53
54                 let wait_time = next_wakeup.saturating_duration_since(
55                     Instant::now());
56                 if wait_time > Duration::from_millis(0) {
57                     state = cvar.wait_timeout(state, wait_time).unwrap(
58                         ) .0;

```

```

57         }
58     }
59 });
60
61     ConcurrentCacheImpl {
62         state,
63         ttl: d,
64         cleaner_thread: Some(cleaner_thread),
65     }
66 }
67
68 fn get(&self, key: &str) -> Option<Arc<String>> {
69     let (lock, _) = &*self.state;
70     let mut state = lock.lock().unwrap();
71
72     if let Some((val, expiry)) = state.map.get(key) {
73         if Instant::now() < *expiry {
74             return Some(Arc::clone(val));
75         } else {
76             state.map.remove(key);
77         }
78     }
79     None
80 }
81
82 fn set(&self, key: &str, value: &str) {
83     let (lock, cvar) = &*self.state;
84     let mut state = lock.lock().unwrap();
85     let expiry = Instant::now() + self.ttl;
86     state.map.insert(key.to_string(), (Arc::new(value.to_string()),
87     expiry));
88     cvar.notify_one();
89 }
90
91 impl Drop for ConcurrentCacheImpl {
92     fn drop(&mut self) {
93         let (lock, cvar) = &*self.state;
94         {
95             let mut state = lock.lock().unwrap();
96             state.stop = true;
97             cvar.notify_all();
98         }
99         if let Some(thread) = self.cleaner_thread.take() {
100             let _ = thread.join();
101         }
102     }
103 }

```

Step 4: Testing and Verification

The implementation should be tested using concurrent accesses to `set` and `get`. Proper memory cleanup is verified by ensuring the background thread is joined during the `Drop` phase. Tests should assert that an entry retrieved after the `Duration` returns `None`.